

Java OOPs (Object-Oriented Programming System) Notes

Essential Concepts for Interview Preparation

Java is a premier object-oriented language used for web, mobile, and enterprise software. Mastering these core principles is vital for technical assessments and real-world development.

1. Class

Definition: A Class is a blueprint or template used to create objects.

Example: `class Student {`

`String name;`

`int age;`

`}`

Real-Life Analogy: A blueprint of a house.

2. Object

Definition: An Object is an instance of a class that contains data and behavior.

Example: `Student s1 = new Student();`

`s1.name = "Rahul";`

`s1.age = 20;`

Real-Life Analogy: An actual house built from the blueprint.

3. Encapsulation

Definition: Encapsulation is the process of wrapping data and methods into a single unit (class) and protecting data using private access.

Example: `class Student {`

`private int age;`

`public void setAge(int age) { this.age = age; }`

`public int getAge() { return age; }`

`}`

Real-Life Analogy: An ATM Machine (internal data and processing are hidden).

4. Abstraction

Definition: Abstraction means hiding implementation details and showing only essential features to the user.

Example: `abstract class Animal {`

`abstract void sound();`

`}`

```
class Dog extends Animal {  
  
    void sound() { System.out.println("Bark"); }  
  
}
```

Real-Life Analogy: Driving a car (you use the steering and pedals without needing to understand engine mechanics).

5. Inheritance

Definition: Inheritance allows one class (child/subclass) to acquire the properties and methods of another class (parent/superclass).

Example:class Animal {

```
    void eat() { System.out.println("Eating..."); }  
  
}
```

```
class Dog extends Animal {
```

```
    void bark() { System.out.println("Barking..."); }  
  
}
```

Real-Life Analogy: A child inheriting physical traits from their parents.

6. Polymorphism

Definition: Polymorphism means "one method, many forms." It allows objects to be treated as instances of their parent class.

Method Overloading (Compile-Time)class Calculator {

```
    int add(int a, int b) { return a + b; }
```

```
    int add(int a, int b, int c) { return a + b + c; }
```

```
}
```

Method Overriding (Run-Time)

```
class Animal {
```

```
    void sound() { System.out.println("Animal Sound"); }
```

```
}
```

```
class Dog extends Animal {
```

```
    @Override
```

```
    void sound() { System.out.println("Dog Barks"); }
```

```
}
```

Real-Life Analogy: A person can be a Student, Employee, or Teacher simultaneously depending on the context.

7. Interface

Definition: An Interface is a contract that contains method declarations which implementing classes must define.

Example:

```
interface Vehicle {
```

```
    void start();
```

```
}
```

```
class Car implements Vehicle {
```

```
    public void start() { System.out.println("Car Started"); }
```

```
}
```

8. Abstract Class

****Definition:**** An Abstract Class is a class that cannot be instantiated and may contain both abstract methods (without bodies) and concrete methods.

****Example:****

```java

```
abstract class Shape {  
  
    abstract void draw();  
  
    void display() { System.out.println("Display Shape"); }  
  
}
```

9. Constructor

Definition: A Constructor is a special method that initializes an object automatically when it is created.

Example:class Student {

```
    Student() { System.out.println("Constructor Called"); }  
  
}
```

10. Method

Definition: A Method is a block of code that performs a specific task.

Example:class Demo {

```
    void hello() { System.out.println("Hello Java"); }  
  
}
```

OOPS Interview Summary Table

OOP Concept	One-Line Definition	Real-Life Example
Class	Blueprint used to create objects.	House Blueprint
Object	Instance of a class.	Actual House
Encapsulation	Wrapping data and methods together (Data Hiding).	ATM Machine
Abstraction	Showing essential features while hiding implementation.	Car Driving
Inheritance	One class acquires properties of another.	Parent → Child
Polymorphism	One method behaves in many forms.	Person with different roles
Interface	Contract that implementing classes must follow.	Remote Control
Abstract Class	Template that cannot be instantiated.	Vehicle Template
Constructor	Special method called automatically on creation.	Initial phone setup
Method	Block of code that performs a task.	Calculator function

High-Frequency Interview Topics

The Four Pillars of OOP

1. **Encapsulation:** Focuses on Data Hiding.
2. **Abstraction:** Focuses on Hiding Implementation.
3. **Inheritance:** Focuses on Code Reusability.
4. **Polymorphism:** Focuses on "One Interface, Many Implementations."

Interface vs. Abstract Class

1. Interface (The "What")

- **Definition:** Defines what should be done, but not how it is done.
- **Real-Life Example: A TV Remote.** The remote has buttons like "Power On" and "Volume Up." It defines the operations, but different TVs (Sony, LG, Samsung) implement the internal circuitry differently.

2. Abstract Class (The "Commonality")

- **Definition:** Provides common functionality and allows child classes to implement specific behavior.
- **Real-Life Example: A Vehicle.** Every vehicle has common methods like `start()` or `refuel()`, but the method `drive()` is abstract because a car drives with 4 wheels while a bike drives with 2.